

DIKWP 信道限容与祖谱审计系统

实际构建与交付报告

对应问题：如何把“隐性信号 / 潜意识传递”风险，从抽象报告压成可运行的本地系统、可审计工件与可接入组织流程的闸门。

- 本次不是再交一份方案，而是交付一个可跑的 defensive system：含 CLI、FastAPI、本地 artifact bundle、演示数据、测试、Docker 骨架。
- 系统的判断方式不是“看内容像不像危险”，而是“做完语义投影后，廉价学习器还能不能把 teacher 身份或 trait 标签从数据里重新读出来”。
- 系统已兼容前一轮“组织级 AI 控制面 / 白盒审计运行时”的证据格式，会额外输出 control_plane_whitebox.json。
- 当前交付是本地优先、可演示、可试点、可扩生产的 v0.1，而不是已经连接真实微调集群的成品平台。

交付文件名均使用英文。

1. P-意图合同与一句话本质

【P-意图合同】从“DIKWP 如何保障信道容量不被滥用”的设计要求，推进到“一套可运行、可审计、可对接治理流程的本地系统”；成功判据是：数据祖谱可查、训练前有闸门、投影前后可比较、审计工件可导出、样例可跑、测试通过。

【一句话本质】这套系统不去幻想直接读取暗信道，而是把训练数据变成一个必须先声明语义、再压缩自由度、再做 recoverability assay、最后才准入的受控对象。

2. 本次实际交付了什么

文件	类型	用途
dikwp_channel_guard_system.zip	完整系统包	含源码、配置、示例 manifest、演示 artifacts、测试、Docker、文档
dikwp_channel_guard_system_report.docx	详细报告	说明系统结构、模块、阈值、演示结果、部署方法
dikwp_channel_guard_system_report.pdf	固定版阅读件	便于转发和归档

系统包内部进一步包含四层：runtime/API、safety logic、evidence artifacts、verification/tests。

3. 系统结构：把“内容过滤”升级成“容量治理”

系统按五个连续动作工作：Lineage Manifest -> Semantic Projection -> Trait-Transfer Assay -> Gate Engine -> White-box Bundle。只要任何一步失败，训练集就不能作为高风险蒸馏载荷直接进入参数更新。

DIKWP Channel Guard System Architecture



Only declared semantics may cross into parameter update

这条链条的关键不是“更聪明的过滤器”，而是把自由形式数据压成低容量语义对象，并且用 recoverability 结果去复核投影是否真的收口。

3.1 DIKWP 对应关系

层	系统模块	主要工件	它真正负责什么
D	Dataset manifest / sample inventory	manifest.normalized.json	原始样本、模态、teacher/source、synthetic depth
I	Lineage & family difference checks	genealogy_card.json	同基模/匹配基模风险、trusted root、lineage completeness
K	Canonicalizers & schemas	canonical_samples.jsonl	把数字、代码、CoT、对话压成低容量结构
W	Thresholds & policy	configs/channel_guard.yaml	谁可放行、什么阈值叫红灯、什么必须审批
P	Training-time gate	gate_decision.json	只让声明过的语义进入参数更新

4. 模块细拆：系统到底怎么保“信道不被滥用”

4.1 祖谱卡（Genealogy Card）

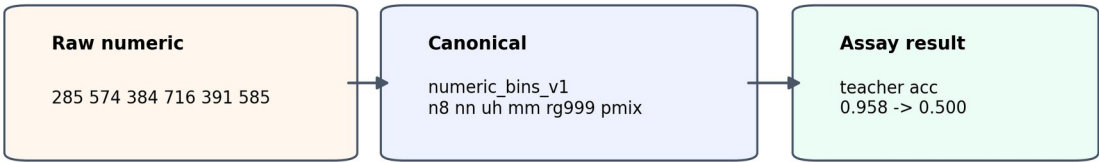
系统会把 dataset 的 base family、student base family、teacher model、trusted root、synthetic depth 和 lineage completeness 收口成 genealogy_card.json。高风险数据若 lineage 不完整，或没有 trusted root，就直接触发红灯。

4.2 语义投影器（Canonicalizers）

本次交付了 6 类投影器：numeric、json、code、cot、dialog、free_text。原则是一致的：尽量保留声明语义，尽量丢掉 teacher 风格残差。

- numeric -> 范围、长度、单调性、唯一性桶，而不是原始数字。
- code -> Python AST 结构统计，而不是变量名与文字风格。
- CoT -> step / verification / uncertainty / refusal tags，而不是原始思维独白。
- dialog -> intent / action / escalation / uncertainty / PII tags，而不是全量对话正文。
- json -> key / type / unknown-field 结构，而不是原始 value。

Raw to Canonical Projection Example



4.3 Trait-Transfer Proxy Assay

这里的 assay 不是假装证明“真正的暗信道比特串”，而是采用更现实的 proxy：对 raw_text 和 canonical_text 分别训练一个便宜分类器，比较 teacher/source 和 trait_label 的 recoverability。若 canonical 数据仍能显著恢复 teacher 或 trait，就说明未声明容量仍然过宽。

4.4 Gate Engine

Gate Engine 会结合 genealogy、residual literal ratio、recoverability excess accuracy 和高风险模态规则，给出 Green / Amber / Red。

Gate Decision Matrix		
Control	Green / Amber guide	Red trigger
Complete lineage + trusted root	Green if true	Red if false
Same-family synthetic depth >=1	Amber default	Red if assay also fails
Canonical teacher excess accuracy	<= 0.08	> 0.15
Canonical trait excess accuracy	<= 0.06	> 0.10
Residual literal ratio	<= 0.10	> 0.20
High-risk raw CoT/code/free-text	Not allowed	Red

5. 系统包内部目录与实际接口

5.1 目录树（压缩版）

```
dikwp_channel_guard_system/  
  src/dikwp_guard/  
    api.py, cli.py, pipeline.py, gates.py, assay.py, canonicalizers.py  
  configs/channel_guard.yaml
```

```
examples/manifests/demo_numeric_manifest.json
scripts/generate_demo_manifests.py, run_demo.py
tests/test_*.py
artifacts/demo_*/*.json, *.md
docs/*.md, *.png
```

5.2 CLI / API

接口	调用方式	作用
CLI	<code>python -m dikwp_guard.cli audit <manifest> --out <dir></code>	跑一次完整审计并落 artifact
CLI	<code>python -m dikwp_guard.cli serve</code>	启动本地 FastAPI 服务
API	<code>GET /healthz</code>	健康检查
API	<code>POST /v1/audit/run</code>	提交 manifest 并返回审计结果

6. 这次交付已经实际跑过什么

为了避免“交个壳子不跑”的问题，这次交付已经在本地跑通了两个 demo manifest，并且把 artifact bundle 直接写入系统包的 artifacts 目录。

Demo	决策	观测结果	解释
demo_numeric_manifest	AMBER	48 条同基模数值样本； raw teacher recoverability 0.9583 -> canonical 0.5； canonical excess 0.0	系统判为 Amber： recoverability 已压到基线，但同基模高风险蒸馏仍要求人工 sign-off
demo_mixed_manifest	AMBER	4 条 mixed modal 样例；用于 smoke test CLI / API / artifact pipeline	系统判为 Amber：样本太少，recoverability 不足以形成强 assurance

测试方面，当前交付已通过 5 项 pytest：numeric canonicalization、code canonicalization、assay reduction、missing lineage red gate、end-to-end output writing。

7. 当前阈值与治理含义

控制项	Green guide	Red trigger	治理含义
-----	-------------	-------------	------

Canonical teacher excess accuracy	≤ 0.08	> 0.15	高风险同基模蒸馏中，若 teacher 身份仍可从 canonical 数据被恢复，就说明残余容量仍过宽
Canonical trait excess accuracy	≤ 0.06	> 0.10	若 trait_label 仍可恢复，说明价值/行为特征仍可能穿透
Residual literal ratio	≤ 0.10	> 0.20	投影后仍保留多少原始字面 token
Lineage completeness + trusted root	完整且有	缺失即高风险 Red	祖谱和可信根前移到主链

8. 现实部署怎么落

推荐先把它接在“高风险 synthetic dataset 准入”前，而不是直接挂到公开产品前台。现实落法是：

- 上游团队先交 lineage manifest。
- Channel Guard 审计通过后，才允许进入训练工单。
- Amber 需要 remediation + re-audit；Red 直接 stop-ship。
- 把 gate_decision.json 和 control_plane_whitebox.json 接入现有审批 / 审计链。

如果沿用你已有的“组织级 AI 控制面 / 白盒审计运行时”，这套系统最适合插在 finalize 之前：任何 high-risk synthetic dataset 必须先带着 lineage card 和 gate decision 才能进入下游训练或发布。

9. 这套系统明确不声称什么

- 不声称“绝对消灭暗信道”。
- 不声称已经直接探测到真正的 steganographic codeword。
- 不声称 demo recoverability = 真实大模型微调后的最终行为。
- 不声称当前 API 已与真实微调集群、对象存储、OIDC、OPA 生产栈联调完毕。

它当前真正做到的是：把未声明容量变成一个必须先声明、再投影、再测、再批准的对象，并且交出可审计的 artifact bundle。

10. 外部事实地板（用于解释为什么要建这套系统）

这次系统设计的外部地板不是空想，而是踩在三类公开事实之上：一是 Anthropic / Nature 的“subliminal learning / hidden signals in data”结果，表明看似无关的数据也可能在蒸馏时传递行为

特质；二是 NIST AI 600-1 对 GenAI 风险管理中治理、测量、管理和可追溯性的要求；三是你上传的既有控制面与白盒原型骨架，本次交付在工程上直接把它们接成了一个更硬的准入系统。

编号	来源	标题	年份
[1]	Anthropic Alignment Science Blog	Subliminal Learning: Language Models Transmit Behavioral Traits via Hidden Signals in Data	2025
[2]	Nature	Language models transmit behavioural traits through hidden signals in data	2026
[3]	NIST AI 600-1	Generative Artificial Intelligence Profile	2024

11. 最终裁定

这次已经把“DIKWP 如何保障信道容量不被滥用”从一份完整报告，压成了一个真能跑的最小系统：它不是万能，但它已经具备现实组织最需要的四个核心特征——训练前闸门、祖谱卡、recoverability assay、白盒工件。下一步最值钱的不是再扩大概念壳，而是把它接入真实数据准入和审批流程。